

Université Cadi Ayyad
École Nationale des Sciences Appliquées de Marrakech

Filière Génie Cyber défense et système de
Télécommunications embarquées

LAB 4

Analyse statique d'un APK avec JADX GUI + dex2jar
+ JD-GUI

Réalisé Par :

• LAABOUKI IKRAM

Encadré Par :

PR. MOHAMED LACHGAR

Année universitaire : 2025-2026

1 Introduction

Dans ce travail, nous avons réalisé une analyse statique de l'application Android **UnCrackable Level 1**. L'objectif est d'identifier les vulnérabilités de sécurité sans exécuter l'application, en analysant sa structure interne et son code.

2 Analyse de la structure de l'APK

L'APK a été analysé comme une archive ZIP contenant plusieurs fichiers essentiels.

Observation :

- Présence de AndroidManifest.xml
- Présence de classes.dex
- Présence des ressources (res/)

```
(kali@kali)-[~/APK-Analysis]
$ unzip -l UnCrackable-Level1.apk | head -20
Archive:  UnCrackable-Level1.apk
Length      Date       Time       Name
-----
1648  1980-00-00 00:00  AndroidManifest.xml
1319  1980-00-00 00:00  META-INF/CERT.RSA
1139  1980-00-00 00:00  META-INF/CERT.SF
1077  1980-00-00 00:00  META-INF/MANIFEST.MF
5528  1980-00-00 00:00  classes.dex
1260  1980-00-00 00:00  res/layout/activity_main.xml
456   1980-00-00 00:00  res/menu/menu_main.xml
4751  1980-00-00 00:00  res/mipmap-hdpi-v4/ic_launcher.png
2348  1980-00-00 00:00  res/mipmap-mdpi-v4/ic_launcher.png
7275  1980-00-00 00:00  res/mipmap-xhdpi-v4/ic_launcher.png
14398 1980-00-00 00:00  res/mipmap-xxhdpi-v4/ic_launcher.png
22801 1980-00-00 00:00  res/mipmap-xxxhdpi-v4/ic_launcher.png
2748  1980-00-00 00:00  resources.arsc
-----
66748                                     13 files
```

Analyse : Cette structure confirme qu'il s'agit d'une application Android standard.

3 Analyse du manifest

Le fichier AndroidManifest.xml contient les informations de configuration de l'application.

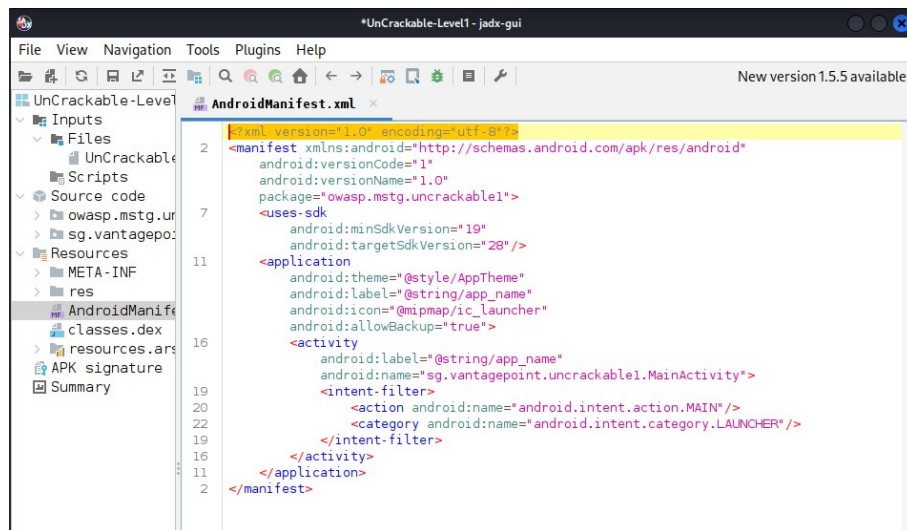
Informations principales :

- Package : owasp.mstg.uncrackable1
- Version : 1.0
- minSdkVersion : 19
- targetSdkVersion : 28

Composants :

- MainActivity (activité principale)

Configuration sensible :
android :allowBackup="true"



4 Permissions et composants exportés

Permissions :

Aucune permission explicite n'a été trouvée dans le fichier AndroidManifest.xml.

Composants exportés :

MainActivity possède un intent-filter (MAIN / LAUNCHER), ce qui la rend accessible comme point d'entrée de l'application.

Analyse : Cette configuration permet la sauvegarde des données de l'application via ADB, ce qui peut entraîner une fuite d'informations sensibles.

5 Analyse du code

5.1 MainActivity

L'analyse du code montre plusieurs mécanismes de protection.

Observations :

- Détection de root
- Détection du mode debug
- Validation du secret côté client

```

1 package sg.vantagepoint.uncrackable1;
2
3 import android.app.Activity;
4 import android.app.AlertDialog;
5 import android.content.DialogInterface;
6 import android.os.Bundle;
7 import android.view.View;
8 import android.widget.EditText;
9 import owasp.mstg.uncrackable1.R;
10 import sg.vantagepoint.a.b;
11 import sg.vantagepoint.a.c;
12
13 /* loaded from: classes.dex */
14 public class MainActivity extends Activity {
15     private void a(String str) {
16         AlertDialog create = new AlertDialog.Builder(this).create();
17         create.setTitle(str);
18         create.setMessage("This is unacceptable. The app is now going to exit.");
19         create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from class: sg.vantagepoint.uncrackable1.MainActivity.1
20             @Override // android.content.DialogInterface.OnClickListener
21             public void onClick(DialogInterface dialogInterface, int i) {
22                 System.exit(0);
23             }
24         });
25         create.setCancelable(false);
26         create.show();
27     }
28
29     @Override // android.app.Activity
30     protected void onCreate(Bundle bundle) {
31         if (c.a() || c.b() || c.c()) {
32             a("Root detected!");
33         }
34         if (b.a(getApplicationContext())) {
35             a("App is debuggable!");
36         }
37         super.onCreate(bundle);
38         setContentView(R.layout.activity_main);
39     }
40
41     public void verify(View view) {
42         String str;
43         String obj = ((EditText) findViewById(R.id.edit_text)).getText().toString();
44         AlertDialog create = new AlertDialog.Builder(this).create();
45         if (a.a(obj)) {
46             create.setTitle("Success!");
47             str = "This is the correct secret.";
48         } else {
49             create.setTitle("Nope...");
50             str = "That's not it. Try again.";
51         }
52         create.setMessage(str);
53         create.setButton(-3, "OK", new DialogInterface.OnClickListener() { // from class: sg.vantagepoint.uncrackable1.MainActivity.2
54             @Override // android.content.DialogInterface.OnClickListener
55             public void onClick(DialogInterface dialogInterface, int i) {
56                 dialogInterface.dismiss();
57             }
58         });
59         create.show();
60     }

```

Analyse : La validation du secret est réalisée localement dans l'application. Cela constitue une vulnérabilité critique car un attaquant peut analyser le code et récupérer le secret sans interaction avec un serveur.

5.2 Analyse cryptographique

L'application utilise un mécanisme de chiffrement basé sur AES.

Observations :

- Clé AES stockée directement dans le code
- Données encodées en Base64

```

1 package sg.vantagepoint.uncrackable1;
2
3 import android.util.Base64;
4 import android.util.Log;
5
6 /* loaded from: classes.dex */
7 public class a {
8     public static boolean a(String str) {
9         byte[] bArr;
10        byte[] bArr2 = new byte[0];
11        try {
12            bArr = sg.vantagepoint.a.a.a(b("8d127684cbc37c17616d806cf50473cc"), Base64.decode("SUJiFc"));
13        } catch (Exception e) {
14            Log.d("CodeCheck", "AES error:" + e.getMessage());
15            bArr = bArr2;
16        }
17        return str.equals(new String(bArr));
18    }
19
20    public static byte[] b(String str) {
21        int length = str.length();
22        byte[] bArr = new byte[length / 2];
23        for (int i = 0; i < length; i += 2) {
24            bArr[i / 2] = (byte) ((Character.digit(str.charAt(i), 16) << 4) + Character.digit(str.charAt(i + 1), 16));
25        }
26        return bArr;
27    }
28 }

```

Analyse : Le stockage de la clé dans le code rend le chiffrement inefficace, car un attaquant peut l'extraire et déchiffrer les données.

6 Analyse des ressources

Le fichier strings.xml contient les chaînes de caractères utilisées dans l'interface.

Observation :

- Texte : "Enter the Secret String"

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3     <string name="action_settings">Uncrackable1</string>
4     <string name="app_name">Uncrackable1</string>
5     <string name="button_verify">Verify</string>
6     <string name="edit_text">Enter the Secret String</string>
7     <string name="thanks">With special thanks to Bernhard Mueller for creating the app. Now maintained</string>
8 </resources>

```

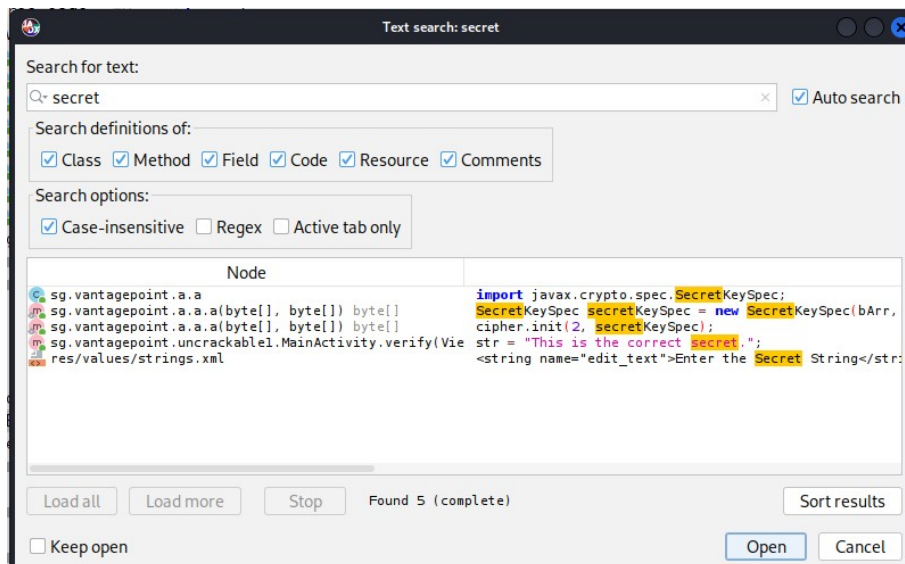
Analyse : Ce message indique que l'utilisateur doit entrer un secret, confirmant la présence d'un mécanisme de validation.

7 Recherche de chaînes sensibles

Une recherche de mots-clés a été effectuée dans le code.

Résultats :

- Présence du mot "secret"
- Détection de debug
- Détection de test-keys
- Aucune URL externe

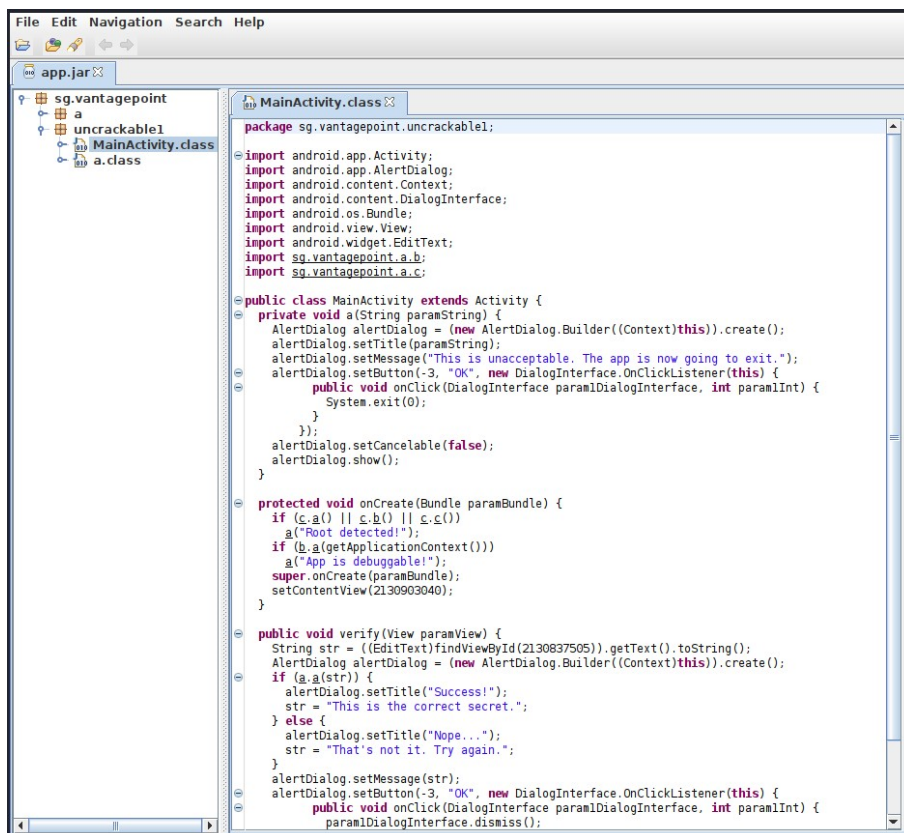


Analyse : Ces résultats montrent que l'application implémente des mécanismes anti-analyse, mais contient également des indices révélant une logique de validation interne.

8 Analyse avec JD-GUI

Le fichier classes.dex a été converti en app.jar et analysé avec JD-GUI.

Observation : Le code obtenu est similaire à celui de JADX.



Analyse : JD-GUI permet une lecture simplifiée du code mais ne fournit pas les ressources Android.

9 Vulnérabilités identifiées

- Validation côté client (**ÉLEVÉ**)
- Clé AES exposée (**ÉLEVÉ**)
- Données récupérables (**ÉLEVÉ**)
- allowBackup activé (**MOYEN**)

10 Conclusion

L'application implémente certaines protections (anti-root, anti-debug), mais reste vulnérable au reverse engineering.

Risque global : ÉLEVÉ

11 Recommandations

- Ne jamais stocker de secrets côté client
- Déplacer la validation vers un serveur sécurisé
- Protéger les clés cryptographiques
- Désactiver allowBackup